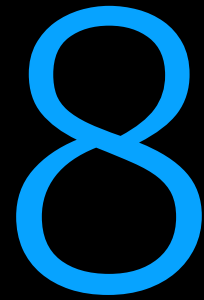


Software Architecture

Compatibility and Coupling



Contents

- Adapters and Wrappers
- Interface Mismatches
- Adapters and Interoperability
- Adapting between Synchronous and Asynchronous Interfaces
- Scaling Adapters to N Interfaces
- Compatibility and Interface Standards
- Coupling Facets
- Binding Times

Compatibility



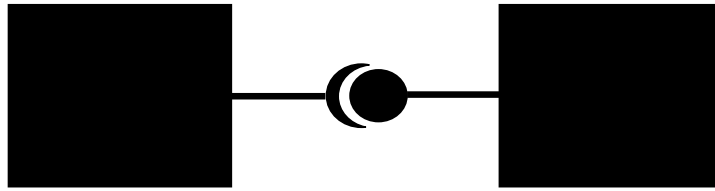
1435mm Standard Gauge
1668mm Iberian Gauge
1524mm Russian Gauge
1000mm RhB (Swiss Alps)

How is it possible for the train to roll on the tracks? The wheels are the point in which the train touches the tracks. So that's the interface element between the rest of the train and the tracks. The assumption behind their compatibility is that the distance between the wheels needs to match the distance between the rails.

There is not only one standard setting for this value: depending on the history and geographic location, the type of train network, there are many different possibilities. The chosen distance is actually arbitrary, an historical accident. It may be due to political decisions to make the trains compatible or incompatible between different train networks. The distance has also an impact on the size and the speed of the trains. Spanish trains have the most comfortable riding experience because they are so much larger. If you are riding a train across the Swiss Alps, there are lots of sharp turns: it pays off to shrink the rails distance so that you can make trains climb to the top of the mountains.

What is the connector in this scenario between the train and tracks? Gravity. It's good to have such force on your side if you want to keep components together.

Compatible Interfaces



- To be connected, component interfaces need to match perfectly

The main assumption that we make when we connect together two components is that they are compatible. In other words, the interfaces connected by the connector need to match, perfectly.

But what is it that keeps the two software components together? The coupling between matching and connected interfaces.

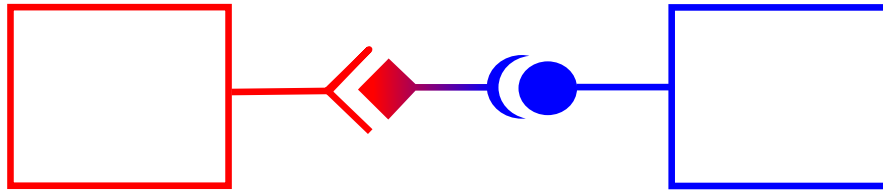
There's an ~~app~~ adapter for that!



You can build a whole industry just by solving interface compatibility issues. Provide people with a solution to connect incompatible interfaces: there is an adapter for that.

You may rightfully wonder if the design of incompatible interfaces is actually intentional. You would not be able to sell any adapters if all interfaces were compatible in the first place.

Adapter



- Adapters help to connect mismatching interfaces
- Adapters deal with transforming the interaction provided by one to the interaction required by the other interface
- Warning: if data or functionality are missing the adaptation may be impossible

When we work with software we assume that the component interfaces have to match perfectly so they can be connected. If you have two components but their interfaces do not match and they are not compatible, then you have a problem.

Sometimes you can solve the problem by building an adapter. The first part of the lecture is going to be about: how do we work with software adapters? and what kind of incompatibilities can be solved with adapters and which ones are impossible?

What is an adapter? It's a specific kind of software connector which is dedicated to solving interface mismatches.

To deal with such mismatches, some kind of data transformation might be needed. As each interface comes with their own data model defining the data types that the interface understands, and can exchange with other interfaces sharing the same data types and structures.

The first function of the adapter is to convert between different data representations, between different formats within the same semantic domain.

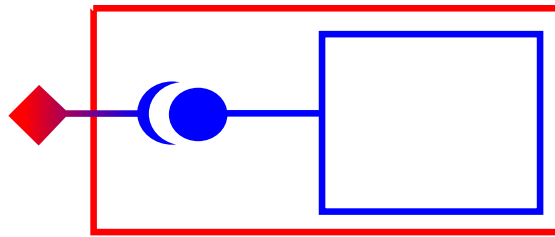
But there is more than just incompatible data representations. There are also different types of interactions. For example, you could have the blue component working with an API based on synchronous calls, and the red component assuming that it depends on a message-based asynchronous interface. Adapters may also need to transform between asynchronous and synchronous interactions.

Is adaptation always possible? Not always: if the data that you depend on, or if the functionality that you require is missing. The adapter cannot find it anywhere within the interface it is trying to adapt. In this case, the adaptation would fail, unless you would overload the adapter with a re-implementation of missing functionality or make it stateful by storing the data which is not found in the original interface.

This goes beyond what an adapter should be dedicated for: it is after all just a connector between two components, rewiring the pins. But if you go back to the hardware example, you will be surprised by the complexity of some of those adapters. Not only are the interface incompatible, but missing implementations have been outsourced into the adapters themselves.

Here is the adapter: with it we are able to connect two components even though their original interfaces are not compatible. Remember: adaptation is not always possible.

Wrapper



- The mismatching end of the adapter is hidden inside a wrapper component
- The adaptation logic is encapsulated within the wrapper and made reusable

What is a wrapper? A wrapper makes the adapter reusable by hiding it inside a compatible interface.

The Blue Interface is the original one, which may be incompatible. The Red Interface is the interface of the wrapper, which allows to directly connect to other components without these components realizing that the component is actually incompatible with what they require.

The red interface is compatible because it wraps the incompatible (blue) one. From the outside, you do not see the old interface, but you can use the adapter directly.

Wrapping is a combination between adaptation and abstraction; adaptation with encapsulation gives you a wrapper.

Mismatch Example

```
id upload(user, image);  
image download(id);  
setTitle(id, title);  
title getTitle(id);  
ids[] list(user);
```

A

```
id upload(user, image, title);  
{user, title, time} getImageMetaData(id);  
image getImageData(id);  
ids[] list();
```

B

Are Interfaces A and B equivalent?

- ✗ Yes (A can replace B and vice-versa)
- ✗ A can be replaced by B
- ✗ B can be replaced by A
- ✓ Not completely

This examples shows you the type of problem you have to deal with when attempting to connect two mismatching interfaces.

Equivalence means that it is possible to write an adapter which transforms one into the other and vice versa. Is one interface replaceable by the other and vice versa? If this is true for both, then they are equivalent.

In general, it's unlikely that a pair of arbitrary interfaces can be considered as equivalent, because there is always going to be some feature that is present in one interface but missing from the other.

It is more likely that it is possible to do the replacement in one direction only. It may be possible to have an adapter from one to the other interface, but not the other way around.

Also if it is not possible to build an adapter for the complete interface, you may be lucky that your component does not require the whole interface but only a subset for which the adaptation is possible.

Consider the example and try to answer the question before we try to write the two adapters.

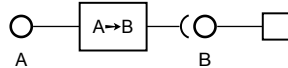
```
id upload(user, image);
image download(id);
setTitle(id, title);
title getTitle(id);
ids[] list(user);
```

A

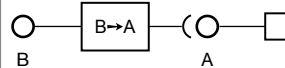
```
id upload(user, image, title);
{user, title, time} getImageMeta-data(id);
image getImageData(id);
ids[] list();
```

B

Partial Wrappers



```
id upload(user, image) {
    return b.upload(user, image, "");
}
image download(id) {
    return b.getImageData(id);
}
setTitle(id, title) {
    ?
}
title getTitle(id) {
    return b.getImageMeta-data(id).title;
}
ids[] list(user) {
    return b.list().filter(user);
}
```



```
id upload(user, image, title) {
    id = a.upload(user, image)
    a.setTitle(id, title)
    return id;
}
{user, title, time} getImageMeta-data(id) {
    return {?, a.getTitle(id), ?}
};
image getImageData(id) {
    return a.download(id);
};
ids[] list() {
    return a.list("*");
};
```

In the example, it's not possible to completely replace one with the other.

Let's start from A to B: we have a component that depends on A and we try to write an adapter that implements A's interface using B's interface.

There is one property offered by A, which has the 'getTitle' and 'setTitle'. While we can map the property read to the 'getImageMeta-data' getter, we cannot find the corresponding feature in B. A workaround would be to re-upload the image every time the title changes, but this would invalidate the image identifier and fill up the B implementation with duplicate images. The adapter would need to keep track of the identifiers associated with duplicate images, thus becoming stateful.

If you switch perspective, and try to go from B to A, then we spot one case in which we have a very broad operation: the 'getImageMeta-data' which returns all the metadata for a certain ID. If you look at what is available to do the same in interface A, we only have the title, so you will be able to return the title, but you will not be able to return the user or the time unless you keep track of those inside the adapter. To do so, you will have to make a stateful adapter that remembers when the image was uploaded as well as which user is associated with it.

Adding state to an adapter introduces redundancy (overlapping information about same item is stored in different places) which leads to potential inconsistency.

Stateless adapters simply map or transform interfaces. Stateful adapters do so but also need to remember the entire history of the interactions with the interface and their life-cycle becomes tightly coupled with the one of the component they are adapted.

If you really need to add state to an adapter, then make it a wrapper, so that you can hope that all interactions with the mismatching component go through the adapter. Bypassing the adapter and using the original interface may render the adapter state inconsistent.

Types of Interface Mismatches

- Cannot be solved with adapter:
 - Missing required feature
- Can be partially solved with adapter:
 - Different data types
- Can be solved with adapter:
 - Same feature with different name/order
 - Operation granularity (one vs. many)
 - Data granularity (one structured parameter vs. many simple parameters; scalar vs. vector)
 - Interaction (e.g., synchronous vs. asynchronous)

In general, if we consider what types of interfaces mismatches are there, we can see that if a required feature is missing, we cannot solve this with an adapter. Required features missing from the provided interface are a big problem, because there is no adapter that will actually solve it unless you're willing to re-implement the missing feature inside the adapter. But that's not the job of the adapter.

There is another case in which we can solve it partially with an adapter if, for example, we have different, but partially overlapping data types. If you have to transform integers into floating point numbers. All integers are floating points, but if you go the other way then you have a rounding problem.

There are other cases in which it's easy to do the adaptation. For example, if you have a different name for the same thing, you just have to rename it. If you have the wrong granularity, it's possible to decompose a large operation by composing the small ones offered by an interface. It's more difficult to do the opposite with a stateless adapter. Mapping many fine-grained operations on top of a coarse-grained interface may require to accumulate all the small calls and then make the big call in one shot. But how do you know when you have enough data to pass on to the coarse grained interface? And what if the last piece of information never comes? The big call will never happen.

Mismatches can be found at the level of operations, events and properties but they can also be at the level of the data models. There can be APIs that work with individual data items and others which work in batch mode with collections of items. It can be more efficient to transfer a whole batch of objects that have the same structure as opposed to having to make calls for every separate object.

Another case that is a very challenging concerns mismatches in the type of interaction. The data is compatible. The operation is exactly the same, but there is still a difference: one interface is synchronous while the other is asynchronous.

Synchronous vs. Asynchronous Interfaces

Synchronous	Asynchronous
Block the caller	<code>f(x, (y) => { })</code>
Easier to program with	<code>emit(x) on(y => { })</code>
Require polling/busy waiting to detect events	Closer to the hardware
<code>y = f(x)</code>	Use callbacks for event notification

Let's compare synchronous and asynchronous interfaces. You can recognize the difference in the code snippets and also if you can see what properties, challenges or constraints each comes with.

In some scenario, it is actually useful to have a synchronous interface, and in other scenarios it is better to have an asynchronous one. So it is not always clear which one to pick.

And as a result, it's unavoidable to have both in the same architecture, and as a consequence you have to be able to bridge this mismatch.

Synchronous interfaces involve calls: you make a call, wait until you get the answer and, as a consequence, you – the caller – are blocked. You will need to wait until the result comes back. Since there is nothing simpler to program than calling a function, they are the easiest to write your code against.

Synchronous interfaces have a little drawback. What if the purpose of the call is to monitor state changes inside the component being called? In other words, you want to detect if an event has happened, then synchronous interfaces they require busy waiting. You have to keep calling to retrieve the latest state until you detect something happened. Or you call once and remained blocked until something happens. Both are extremely inefficient ways to detect events: they keep the caller busy. And they also consume resources on the component being monitored because it's being called all the time.

The interface would work much better if you can just have a callback to notify about the event. And this is the main reason why we have asynchronous interfaces. You can make a call and pass the callback to receive the result. Or you can just use different primitives. You can have event-based or message-based interfaces to receive asynchronous event notifications. Whenever our interfaces encapsulate hardware components, within the components the software will be triggered by interrupts, signaling low-level events. For example, if you have an operating system API that gives you access to the mouse. You will be able to observe the mouse moving by receiving an event that tells you its new position. You can also check what is the current position with a call, but you shouldn't keep asking for the current state all the time if you want to efficiently track the movement.

Synchronous and Asynchronous

How to mix?



Part of our architecture works synchronously (with call-based interfaces) and the other part works asynchronously (with messages or event-based interfaces).

The challenge is: how do we mix them? How do we come up with an adapter? How to connect synchronous and asynchronous interfaces? How can we transform between calls and messages? We are going to use an adapter, which will be different depending on whether we transform one synchronous call into the exchange of two asynchronous messages (the request followed by the response) or vice versa.

Half-Sync/Half-Async

How to connect synchronous (call) and asynchronous (message queue) interfaces?

Use an adapter hiding asynchronous interactions behind a synchronous interface (or viceversa)

Sync2Async adapter: buffer synchronous calls using queues and map them to low-level asynchronous events.

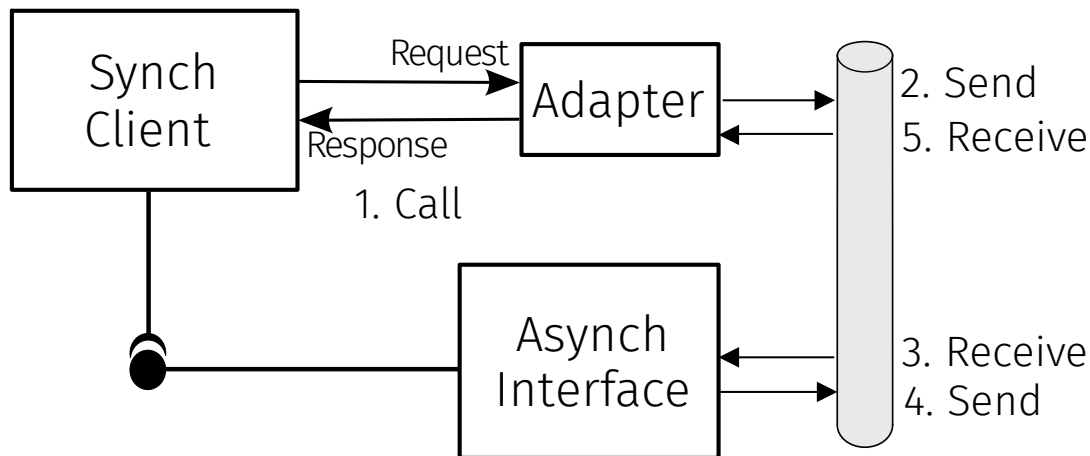
Async2Sync adapter: invoke synchronous interfaces when receiving the corresponding message and emit another message with the call results.

Good news: the mismatch can be solved in both directions.

If we map from synchronous to asynchronous, we have a synchronous call coming in. We have to buffer it using message queues. The incoming call produces an outgoing request message. The adapter listens for the event representing the arrival of the incoming response message. When this happens, the adapter is able to answer the call and unblock the caller.

We can also have an asynchronous clients that needs to be interfaced and connected with synchronous interfaces. Also in this case, the adapter will be listening for inbound messages from the asynchronous client. When the message arrives, we can extract the input parameters needed to make the call. When the call returns, we emit the outbound message with the results back to the original client.

Half-Sync/Half-Async



The adapter converts from synchronous calls to asynchronous messages

We start from the case in which we have a synchronous client which needs to interact with an asynchronous interface, which expects to be connected to a message bus.

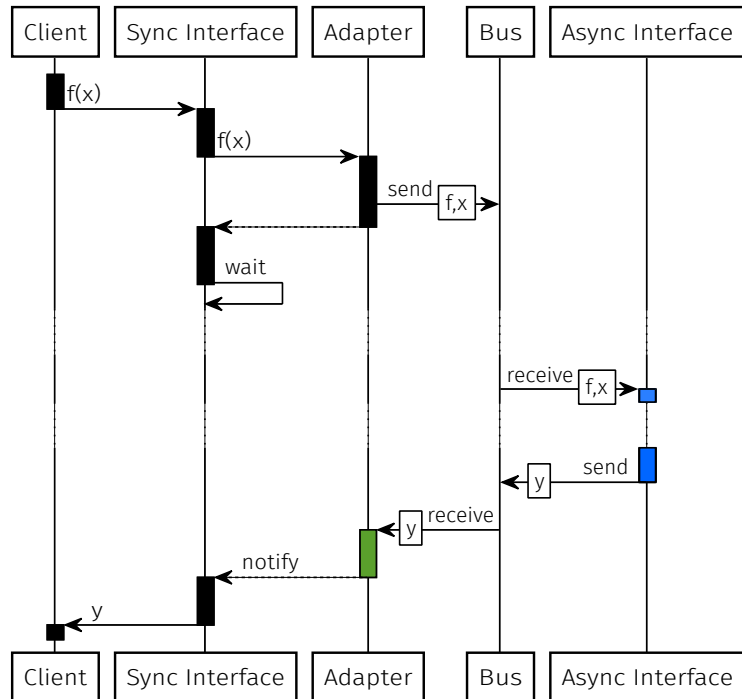
The adapter bridges between two different connectors. On the left side we have a synchronous call. On the right side, the asynchronous message bus. You cannot directly connect them together. By now we have seen that is possible to solve this mismatch with an adapter.

How does the adapter actually work? The interaction begins with the call, intercepted by the adapter. Then the call gets transformed into a request message. The message goes out on the bus. The message is sent on the bus with the call input parameters by the adapter. The asynchronous interface is going to receive it directly from the bus. The asynchronous interface will do whatever it has to do when such messages arrive. Eventually it will produce a response message, which will go out on the bus. The adapter will be listening for it, and will receive it. Once the response has been received, it can be delivered to the synchronous client.

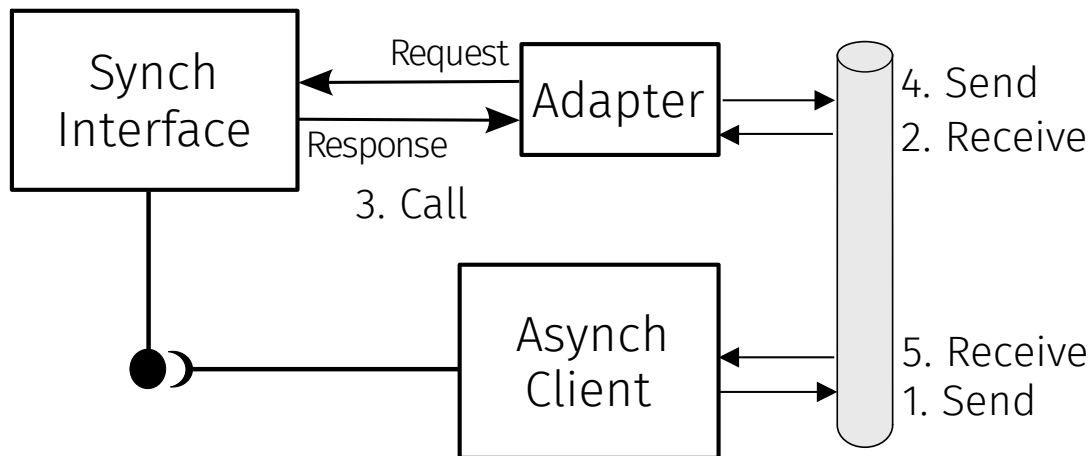
From the perspective of this client it looks exactly as if it was interacting with a synchronous interface. Behind the adapter we have something completely different: a bus with asynchronous interactions.

Sync to Async Adapter

```
f(x) {  
  var result;  
  
  bus.onreceive = (y) => {  
    result = y;  
    notify;  
  }  
  
  bus.send(f,x);  
  
  wait;  
  
  return result;  
}
```



Half-Async/Half-Sync



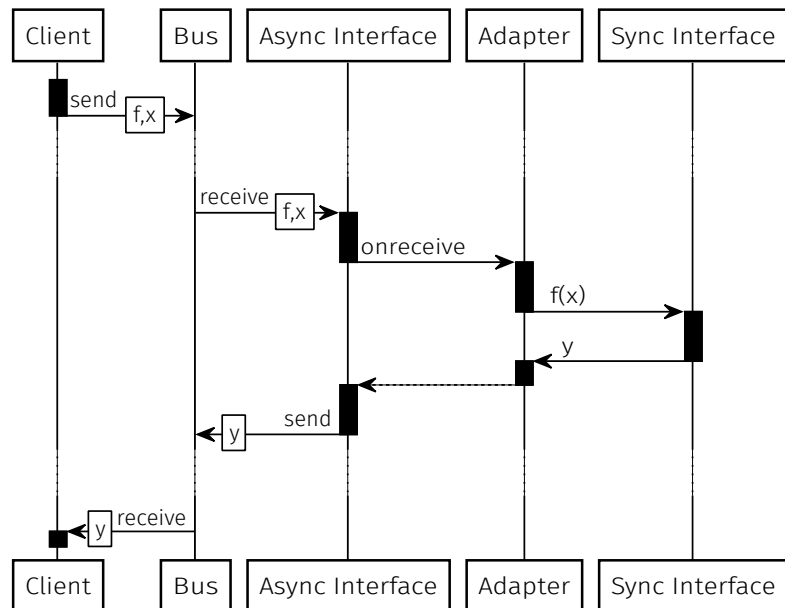
The adapter converts from asynchronous messages to synchronous calls

The structure remains the same. However, the order of the interactions is inverted. The asynchronous client is connected with the bus, which delivers the request message to the adapter. When the message arrives, the adapter converts it into the call. The call is synchronous and blocking the adapter until the result is returned. The adapter packages the call result into the response message, which is sent back to the client.

Async to Sync Adapter

```
//Async Client
f(x,c) {
    bus.onreceive = (y) => {
        c(y);
    }
    bus.send(f,x);
}
```

```
//Async to Sync Adapter
bus.onreceive = (f,x) => {
    var y = f(x);
    bus.send(y);
}
```



Half-Sync/Half-Async

- Benefits of adaptation:
 - Simplify access to low-level asynchronous interfaces (no need to deal with buffers, interrupts, and concurrency)
 - Visibility of interactions (all communication happens through the message bus)
- Disadvantages:
 - Potential performance loss due to layer crossing (data copy, context switching, synchronization)
 - Mapping requires each call to match one pair of messages

To summarize, we cannot build systems only with one type of connector. It's too simplistic to assume we will just make calls everywhere.

You have to be able to mix asynchronous messaging with calls. You have to mix data streams with shared databases and all different kinds of connectors we have seen.

Interfaces are affected by the expectations that they make on the type of connectors that you can use with them.

For example, if you have an interface that is close to some low-level stateful hardware device, you will need to be able to query its current state but also efficiently monitor state changes by listening for events. This can be complicated, so in some scenarios having a simple blocking call that returns when the event is detected can be easier on the client.

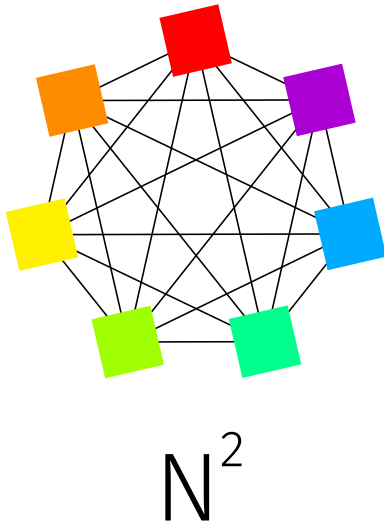
Calls are also more difficult to monitor, unless you can inspect the content of the stack of the current thread. Message buses can easily log and trace messages in transit. This makes it possible to observe at runtime the actual interactions and the communication flow between various components.

Every adaptation has a penalty: You cannot just use an interface directly, but you have to convert. You have to copy data. Maybe you have these locks which you need to 'notify/wait'. A local call is very efficient, while transforming it into four different calls to send and receive two messages that go back and forth in the bus can add significant overhead.

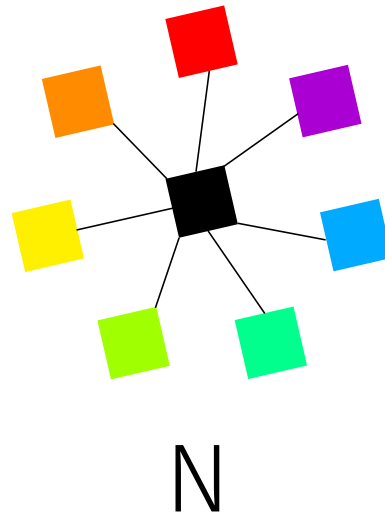
If you look closely, you will be surprised how many times this sync/async adaptation happens along the whole software stack. One asynchronous layer is hidden under a synchronous one, but this one again is turned into an interface which is asynchronous and so forth.

How many Adapters?

One
Point to Point
Adapter



Two Adapters
via intermediate
standard interface



So far we have discussed why we need adapters to connect incompatible interfaces and how we can use encapsulation together with adapters to build wrappers of components so they present an interface that is more compatible than their original one. We have also discussed different types of interface mismatches. Some can be resolved and others are impossible to resolve with adapters. And then we looked at how can we build adapters that help us to connect pairs of incompatible interfaces and in particular focused on the problem of mapping between synchronous and asynchronous interfaces.

In this part we are going to work on: how do we scale the notion of adaptation to work with more than two interfaces? And we will see how standards play an important role in that. We will conclude by looking at the other effect of connecting together two components: they become coupled together. We will discuss different facets of the coupling concept, and in particular we will highlight when coupling is established.

How can we scale adaptation to multiple incompatible components? If you look at this picture, every square has a different color, meaning that has a different interface. Each line connecting the boxes means that it's possible to do the adaptation: it's possible to convert from yellow to orange using a specific adapter. If you have some information inside the yellow component and you would like to transfer it over to the orange one, you can do so because the adapter can transform it from one format to the other.

You can limit the number of adapters but go through multiple adaptations to transfer information along the outer rim, or you can transform directly between the source and destination format. It's always possible to build an interoperable architecture by adding more and more adapters and eventually you reach the format of the level of compatibility that you need. Sometimes this is inefficient because there are many transformations that need to be executed every time you go through one of these edges, there is a cost to pay.

And also there is a risk that the transformation could lose information, which is fine for a direct adapter but could make it challenging to compose multiple adapters.

What if you draw an adapter directly between every pair of components that you want to connect? If you take this to the ultimate consequences with a fully connected graph, you end up with a lot of adapters. Every time you add one more component to your architecture and you want to integrate it with the rest of the system, then you will need to make sure that this component can interact with every other component into your system. Every time you do so, you have to add a larger and larger number of adapters. It doesn't scale as it gets increasingly more expensive with quadratic complexity.

To control the complexity we can introduce a standard: an intermediate interface to which all of the component can be converted to and from. In this case, with only two transformations, we can go from one side to the other. Instead of going directly from yellow to orange, we go from yellow to black and then from black to orange. We can go from this one to the intermediate, and then from the intermediate we can go everywhere else.

This is an important concept that helps you to reduce the complexity of the scalable adaptation problem. You do not have a direct transformation anymore. You have a combination of two transformations, but this is a compromise compared to the earlier scenario in which you had to go through a transformation all around the outer rim of the circle and use up to $N/2$ combined adapters. So here we are in the middle. This is sometimes called a hub and spoke architecture. We go through the center so that we can convert to the standard, and then we have access to everything else. If we want to add one more component to the system like before, then it's much less expensive: all we have to do is write an adapter that converts between the new component interface and the standard and vice versa, as opposed to building a mapping with every other existing interface.

Scaling Adapters with N Interfaces

How to enable communication between many different heterogeneous interfaces?

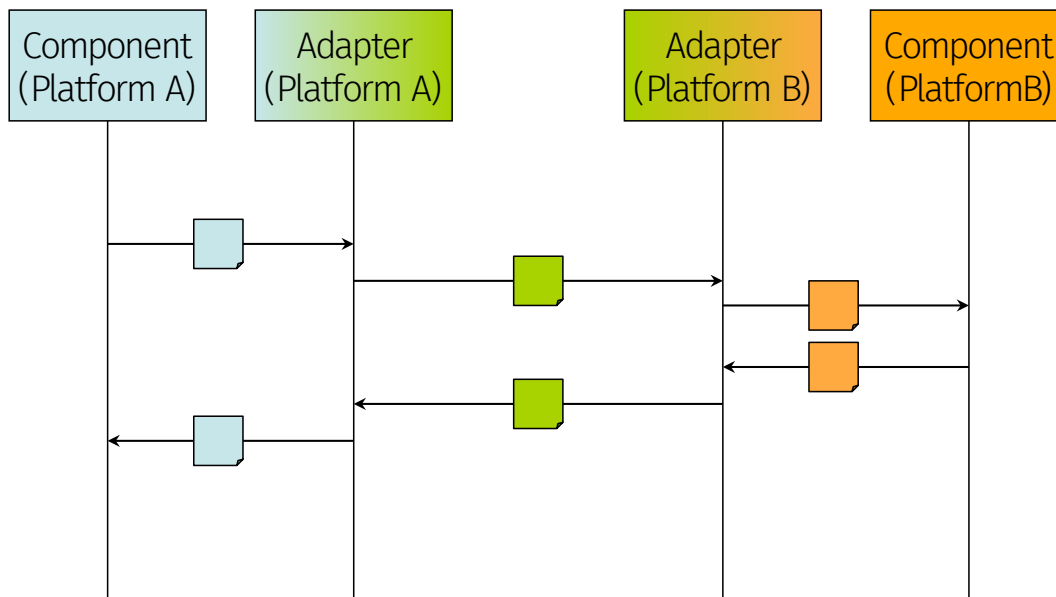
Use a standardized intermediate representation and communication protocol

Before they can be connected, incompatible components need to be wrapped with adapters. These adapters enable interoperability as they implement a mapping to common means of interaction.

If we take a heterogenous system with many different interfaces and we want all of these to be compatible and, more precisely, interoperable, the solution is to first develop a common standard; then build adapters which translate every local interface to the common standard and the interoperability problem is solved.

How does such standard emerge? How is it possible to find such a common representation? It sounds deceptively easy, but it's difficult to achieve in practice: Do you seek the minimum common denominator? Does the standard cover the intersection (to make sure all mappings are feasible) or the union so that every interface will be fully covered and fully represented, but some adapters may disregard or need to provide defaults for some data elements which are missing from the local interface.

Composing Adapters



Components of different platforms can interoperate through adapters mapping the internal message format to a common representation and interaction style

If we look at this pattern from a process viewpoint, the interaction between mismatching interfaces happens through a two step transformation. First, from the local platform, from the local language, from the local representation, we go through an adapter that converts it to the standard (shown in green). Then we do the reverse.

Such transformations have to work in both directions: from local to global, and from global to the other local. This is the minimum requirement for mono directional interactions. If we need a reply to go back in the opposite direction, then we need adapters able to perform the inverse transformations.

One or Two Adapters?

- Why two adapters?
 - For Point to Point solutions one adapter is enough (direct mapping between two interfaces, message translator)
 - In general, when integrating among N different heterogeneous interfaces, the total number of adapters is reduced by using an intermediate representation (or canonical data model)

In case you have to do a quick integration between two components, you don't have to come up with a standard. This is called a point to point integration, where you just design a direct mapping with one adapter.

In general, if you foresee the need to scale the system to a larger number of heterogeneous interface, it pays off to invest into a common standard so that you can later keep the cost of further growing the system under control.

Sometimes this standard is called intermediate representation, or canonical data model. You can find these standards in any kind of integration architecture spanning thousands of different applications, which work together because they agree on a common canonical data model.

You find it as well inside the architecture of a compiler pipeline. First you parse the input language, the result is an abstract syntax tree (AST), which works like an intermediate representation, as different stages of the pipeline operate on it performing different transformations, optimizations, and code generation for specific backends. Having such intermediate representation, separates (or de-couples) the parser of the input language from the back-end emitting the target language. And it even makes it possible to generate code for different processor instruction sets without having to change the parser.

Reusable Adapters and Performance

- Adapters do not have to be re-implemented for each component, but can be generalized and shared among all components of a given interface type
- This pattern helps to isolate the complexity of having to deal with interface differences when building each component. The adapters make the heterogeneity transparent.
- Warning: performance may suffer due to the overhead of applying potentially complex transformations at each adapter (better to design compatible interfaces if possible)

If you target a standard, adapters can become reusable. You can generalize these type of transformations between common formats so that as long as you can map your local interface to the standard, then there can be many different reusable transformations that you can use to connect with other standards.

Be aware: by resorting to one or more combined adapters, there is a performance penalty because the more layers that you have to traverse, the more transformations or adaptation layers you have to inject, the longer it will take for the interaction.

As a design goal, it is much better to try to be compatible in the first place. After all, adaptation is always a backup strategy to recover compatibility, but if you can design something to be compatible in the first place, by all means do it.



It's always possible to attempt to come up with a universal standard that fully covers all the previous use cases partially embedded in existing standards.

As a consequence, here is yet one more standard.

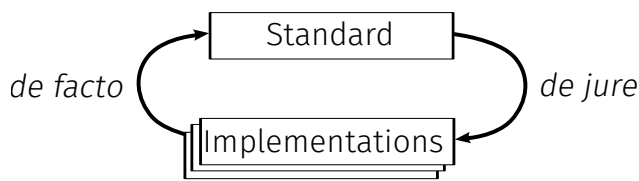
And that's how standard proliferate.

Typically the young developer, or the inexperienced architect will attempt to do that: create a new standard. You – as a student – haven't seen this happen before in our industry, but after you get a bit more experience you will see that this standardization cycle happens all the time.

People try to come up with better standards. This is progress. But for every new standard, we seem never to be able to get rid of the legacy.

This is why I would like now to open a little parenthesis on where standards come from. As you may have noticed, standards are particularly important when we assess the compatibility of component interfaces.

On Standards



- Standard compliant component interfaces are compatible and their implementation replaceable
- Component vendors need to comply with standards to get to the customers depending on them
- Claims of standard compliance should be tested and independently certified

If an interfaces is standard compliant, it means that clients that depend on the interface being compatible with the standard will be able to connect to the interface. Standard compliance makes life easier for the clients which needs to interact with your interface. And at the same time you also make life easier for clients to replace the implementation. The interface remains the same, it's standard. The implementation can change without affecting the clients.

We can observe a fundamental tension between vendors competing to give you the best implementation for the standard, and customers having a low cost in switching between different implementation which are standard compliant and thus easy to replace.

For vendors, there is always the temptation to go beyond the standard to offer components that yes fulfilled the standard interface, but then offer you extra proprietary interfaces. These may fill the gaps of the standard (with useful non-standard features) or are just designed to be more convenient to use than the basic standard.

Once the clients are hooked, they start to depend on the standard as well as on the non-standard part of the interface. This is the vendor lock-in problem. We originally choose this component due to its standard-compliant interface, but then we started to use and depend on the other features so much that we are no longer able to replace it. This ensures the future of the vendor of the specific implementation, who has now a pool of captive clients.

Whenever you see a label on a component stating compliance with a certain interface standard, you shouldn't believe this claim. It can be just a marketing ploy. As you review a checklist of features, it is cheap to add a mark under standard compliance, but more expensive to provide a test suite or have an independent certification of such label. If you don't test it, later on you might encounter some surprises, for example, discover that the standard was only partially supported and that the exact feature you need has not yet been implemented.

On Standards



Which is first, the standard interface or its compliant implementation?

- De facto standards promote existing proven and successful implementations (winner takes all, first mover advantage)
- De jure standards may still need to be implemented (design by committee)

Where do standards come from? This is like a "chicken and egg" problem. Do you start from the standard and then you implement it? Or would you rather find some existing proven implementation that becomes standard?

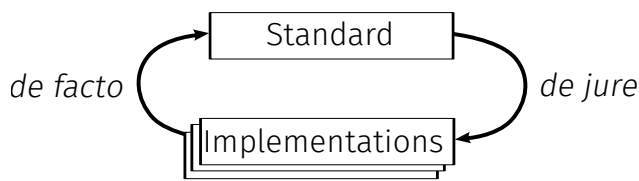
Here we distinguish: de facto vs. de jure standards. If we start from a standard specification and build one or more implementations from it, we have a de jure standard. Sometimes there is only one implementation that is so fast to the market and so powerful, so successful, that becomes – even if there's no formal standard – a de facto standard. Everyone knows that in practice this is the only commonly accepted way to solve that problem.

Being the original developer of a de facto standard gives you first mover advantage and control over who is allowed to implement it after you and who is allowed to actually use it.

What if we implement a standard only after it has been specified? This is a bit risky: how can one standardize something without knowing yet how to make it work? And how can you come up with a design for your interface that is worthy of standardization if no implementation already exists? There is a risk that if you have too many people around the table you get what is called design by committee. The result often is sub-standard in terms of quality (e.g., lack of simplicity) so there is uncertainty on whether it will actually be feasible to implement in a timely manner.

After the initial version, standards typically iterate by cleaning up the specification by incorporating feedback from attempts at its implementation. Committees will standardize, developers will implement, feedback to clarify and improve the standard will be provided. Not necessarily in this order.

On Standards



Who owns the standard?

- Open Standards: everyone can participate and implement them
- Closed Standards: the owner controls who can implement them, competitors usually not allowed.

Standards may have an open nature. For example, if there is an open source, reference implementation and everybody can fork it, improve it and integrate it into their products. With an open standard, everybody shares its "intellectual property" and can participate in a standardization process based on "rough consensus and working code". When the standard is about to be released, there is the possibility for anybody to comment and provide feedback, and vote to finalize it. Also, there is no limit to who can actually implement it.

The Alternative is to have a closed standard where there is a clear ownership of the technology and the interface design, as well as strict control over who is allowed to implement it. If you want to re-implement a certain API, you have to get permission first. And to get it you may have to pay some licensing fees. And if you are a direct competitor, maybe you're not going to be allowed.

There was once a big controversy going on between Oracle and Google concerning whether it's legal for Google to give an alternative implementation of interface standard called: 'Java API'. Google has reimplemented it for their Android operating system, but Oracle has bought it from Sun. While Sun was promoting Java, they took an open standardization strategy. But after Oracle bought it, the strategy changed, and this gave rise to a legal battle about whether it's possible to actually have multiple implementations for an interface unless the owner of the interface allows it. And this can have a big impact on the future of the software industry, because if you can control a software API and you can use the law to limit who is allowed to implement it, this gives you significant monopoly power. The U.S. Supreme Court eventually declared that Google's usage fell well within their rights to do so. Now if only someone would try to provide alternative implementations of popular Web APIs such as the one of Twitter and Facebook.

On Standards



Who writes standards?

- IETF, W3C, OASIS, IEEE, OMG
- ISO
- SWIFT (Financial Messaging)

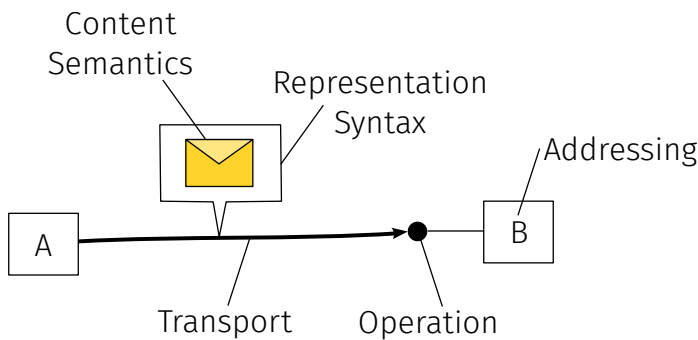
Where do standards come from? They come from standard organizations, like ISO: the International Organization for Standardization. Within the information technology sector, we have standards that are related to networking, hardware, modeling notations, programming languages, as well as software interfaces.

I also added the SWIFT standard to the list in case you're interested about banking and financial messaging. That's just an example of a domain specific standard for ensuring the compatibility of a certain class of applications.

This should give you an idea of how compatibility is achieved. Ultimately compatibility comes from the designer providing interfaces agreeing with the developers who will be consuming them.

One way to reach and encode the agreement is to vote on a standard spelling it out explicitly. If we focus on interfaces, there are several aspects that can be standardized.

Standard Software Interfaces



When you would like to successfully connect your component with the interface of a separate component, you need to achieve interoperability between them with or without an adapter. You have to first make sure that the two parties can communicate. Whatever connector you choose, you will need to make sure that it is possible to transfer some data from A to B. How can the communication work?

First, there needs to be an agreement on the type of transport protocol. If you're deploying A and B in a distributed environment, the communication happens through the network. If they are co-located, then we know that we can use other kinds of connectors like for example shared memory. In this case the transport is a bit simpler, but no matter what type of transport you use, you are going to exchange some data that has certain representation format, using a certain syntax.

There has to be an agreement between the two sides about which syntax they intend to use. When A writes out the message as a sequence of bits or bytes across the transport the bits appear on the other side and then you reconstruct a message carrying the same content.

Once you have exchanged the bits, then you have to give them a structure so that their semantics can be understood.

For example, the recipient has to understand that a messages about a payment just arrived: this is the amount of money, and this is the currency. So you have to assume that both sides agree not only on how do they represent the amount of money (using integers or floating or some other way to encode financial quantities) as well as the currency. Maybe it's just a string, but you don't have to come up with your own possible expected currency values, as there are standards listing all possible currencies that you can pick and choose.

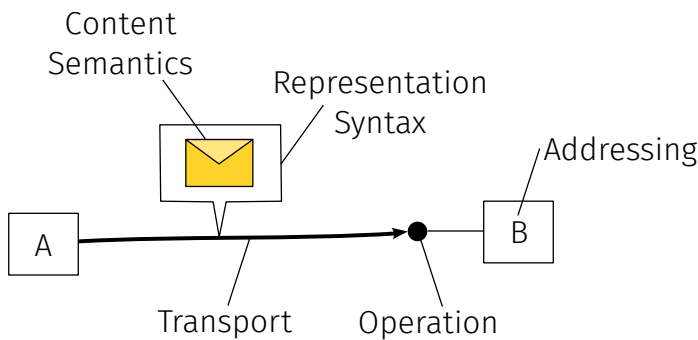
If both parties cannot agree on which standard works best, then you would need an adapter to convert between different formats and mismatching data models.

You want to connect components to invoke operations so the components can call each other. And the set of operations that you can choose from is also a good candidate for standardization. This way there would be an agreement on what is the set of operations that the interface provides, as well as what each operation actually means. The same can be extended to all the interface features: properties, events and the data model.

You also can have a standard way of locating or addressing the interface that you want to interact with. So we need to be able to identify it. We need to know where it is, and we need to know how to reach it.

These are all various aspects, which standardization efforts have been devoted to. As a result, there is a whole technology landscape related to how to construct interoperable and compatible software interfaces. Let's use this as a map to position, classify and compare different tools.

Standard Software Interfaces



Representation Format (Meta-Model)		Data Model
Plain Text	XML	RSS, HTML, SVG, SOAP
	JSON	JSON-LD, GeoJSON
Binary	Protobuf	

Regarding the representation of the data being exchanged across, the first decision that you have to make is whether the data will be sent over the network as text or as binary. Sending text messages, which can be read by a text editor, doesn't necessarily make them human readable, but at least they can be copy and pasted into an email, for example. Or if you choose binary, you can use all the bits (increase the information density of the message) while making it more difficult to view and debug without custom tools.

If you prefer to stay with a readable format, a bit less dense and less efficient in terms of bandwidth, and then you can choose for example between JavaScript object notation (JSON) or the extensible markup language.

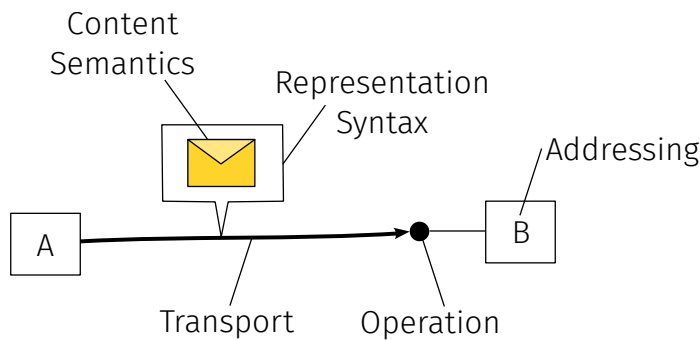
The valuable property of these formats is that they prescribe a syntax but they don't assume anything about the actual structure of the messages that you are going to send following their syntax.

That's the reason why XML has an X because its syntax is used to define other markup languages by fixing the set of tags and their semantics. So from XML we can have HTML (for Web pages), SVG (for vector graphic images), SOAP (for messages), WSDL (for service interface descriptions), RSS feeds (for event logs). These are all specific formats that once you choose to go with XML, then you still need to make another decision which will set the exact type of XML document to be exchanged.

The same is true for JSON, which gives a more lightweight representation for the structure of objects and you can add more assumptions about the field names, or the convention that used to name the fields. This way you can obtain, for example, the GeoJSON format to exchange your geo-located datasets. Or JSON-LD, to represent linked data, with documents that carry references to other documents.

This little decision tree should help you to get started with the interface design decision: How am I going to pick a representation syntax? Before you try to define your own, be aware that there are many standards to choose from. Picking one of them, would make it much easier for others to send messages to your interface, or to read information your interface provides them.

Standard Software Interfaces

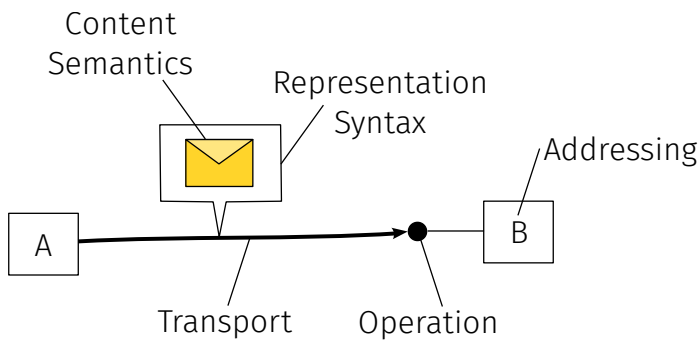


- Operations
 - HTTP Methods (GET, HEAD, PUT, DELETE, POST, PATCH, OPTIONS)

Just one brief example about the operations, showing the most widely successful example of standardizing the set of operations that you can perform on a certain interface. The methods that are part of the hypertext transfer protocol (HTTP) offer a limited, but clearly distinct, set of operations that any Web resource can perform. Anywhere on the Web you will find billions of different resources, which let you read their whole state (GET), or just a subset (HEAD). Some also support updates (PUT) and can be deleted. You can call the resource and perform arbitrary computations (POST). You can also incrementally modify the current state (PATCH).

Additionally, you can perform reflection: ask the interface what is it that you can do with it. OPTIONS is a meta method, which tells you which subset of the previously mentioned methods this particular resource can perform. So you don't have to assume in advance which operations are actually there, but you can discover them dynamically.

Standard Software Interfaces



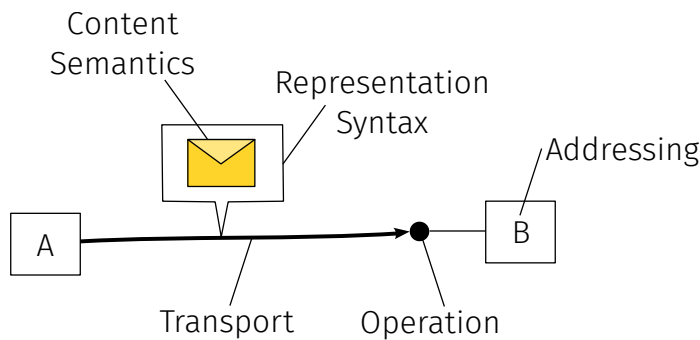
- Protocols

- HTTP (Hyper Text Transfer Protocol)
- SMTP (Simple Mail Transfer Protocol)
- MQTT (Message Queue Telemetry Transport)
- AMQP (Advanced Message Queue Protocol)
- XMPP (Extensible Messaging and Presence Protocol)

The chosen transport protocol helps to exchange data from A to B. Are you going to work with sockets? or can you choose a higher level protocol? So here are a few example protocols that on the networking stack are above TCP or UDP. If you are interacting with a remote Web API, you will probably use the HTTP protocol. There was a time in which people were trying to also use the email protocol (SMTP) to do the same. Then, other, more appropriate, messaging protocols emerged like MQTT or AMQP. This is also a space where you find many proprietary or de-facto standards for transporting messages between two components. Usually it takes decades before the dust settles down and people have found an agreement on which is the winner protocol.

Your choice may depend on what kind of device you plan to deploy your software on. Will the device be part of the Internet of Things? Or the Web of Things? Will it be a cloud-based service? Each of these comes with different presumptive transport protocols, the kind of protocol you would be expected to choose by default.

Standard Software Interfaces



- Addressing
 - URI (Uniform Resource Identifier)
 - UUID (Universally Unique Identifier)
 - DNS (Domain Name System)
 - IPv4, IPv6

Another important aspect is about: where do we find the interface? how do we address elements found within the component interface?

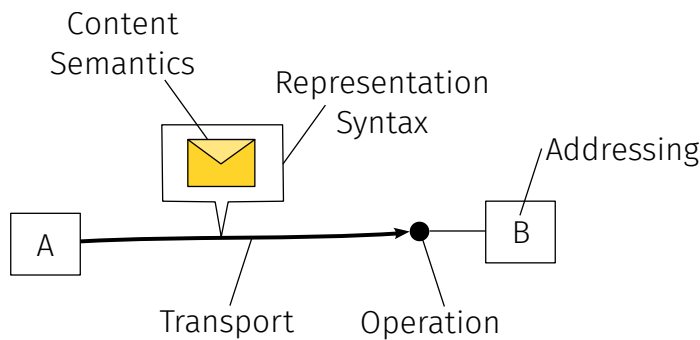
To solve these problems, we can use again networking standards. For example, IP addresses: they have run out for IP version 4, but we can have a bigger address space with IP version 6. We can then use a standard registry for transforming symbolic addresses into the numeric IP addresses: the Domain Name System (DNS).

We can choose to take advantage of Web technology. For example, URI and URI schemes help to invent or reuse powerful mechanisms to structure address identifiers.

Another problem related with addressing is: who is responsible for producing new addresses? Who is managing the addresses? And who is giving names to things? And if you work with the UUID scheme then it is possible to decentralized that operation. All the components in your system can come up with unique identifiers without having to agree beforehand. This independence is more difficult to achieve, for example, with the DNS, which is more hierarchical, but still depending on top level domains, where centralized control over how addresses are handed out is provided.

If you think about Java and Java packages, how are you naming them? Basically you're following a convention to use the reverse DNS symbolic address. If you own this domain, you can name your components after your domain. Noone else should be able to do the same.

Standard Software Interfaces



- Interface Description (Metadata)
 - Schema Definition Languages (XML Schema, JSON Schema)
 - Interface Description Languages (WSDL, Open API, Async API)
 - Data Semantics (schema.org)

There is another level which is about making sure that we can agree on how to describe the interface. It's not about the content of the interface itself, but it's about the languages and the tools that we use to model and represent what is inside the interface.

To do so, we can benefit from standard data management technologies. Schema languages describe a data model: What information does this software interface consume or produce? Just specify the schema. Even though you're not necessarily going to write queries for that schema or store data according to the schema, but you have a schema that tells you: given a certain message, does the content of the message fits within the expected schema? If the message doesn't comply within this schema, it can be filtered, bounced back, or simply discarded and ignored.

In the same way that you work with the type system to compile your code and statically check the validity of object structures based on their classes. Here we can check the messages as they arrive and validate them whether they fit within the constraints of the corresponding schema types.

The schema is only the data modeling part of your interface. The interface, in addition to the data model, also has the operations, properties and events. Those are the focus of interface description languages (IDLs). For example, OpenAPI for Web based APIs or the new AsyncAPI, focusing on message-based interfaces.

You can further constrain messages by specifying not only their syntax, structure but also semantics of the content. This way who is trying to use your interface would know what the data actually means. And the purpose of each operation. In addition to the plethora of Semantic Web languages, it is worth looking at the conventions followed by schema.org.

If you're interested about this, you can follow these pointers to find examples of what it means to describe interfaces, including their semantics.



What is the effect of connecting together two components? We couple them together. There is a benefit because connected components can interact, they can exchange information and communicate, they can transfer control and coordinate their work. They can call each other.

But there's also some other less positive effects. For example, they depend on each other: what could possibly go wrong? If you need a component and the component is not there, maybe there is some consequences for you: you could have cascading failures, propagating along the fault lines of your architecture, induced by coupling.

What if the other side is going to change? You make a change on one component (either the implementation or its interface). Since you are connected and coupled to it, is this change going to affect you? It depends. If you simply touched the implementation, you have a chance to control the impact of the change. If you modified or removed existing features of an interface, then you are looking for trouble.

Highlight the code making these assumptions

Representation
Format

Location
Address

Content
Semantics

```
IPHostEntry hostInfo = Dns.GetHostByName("asq.click");
IPAddress address = hostInfo.AddressList[0];
IPEndPoint endpoint = new IPEndPoint(address, 8888);
Socket socket = new Socket(address.AddressFamily,
    SocketType.Stream, ProtocolType.Tcp);
socket.Connect(endpoint);

Solution answer = new Solution("asq.click", 1);

byte[] message = answer.getBytes();

socket.Send(message);

socket.Close();
```

I would like to ask you if you can apply what you understood about coupling to highlight in these concrete examples the assumptions that we make about the representation format, the address of the component, as well as the semantics of the information that we exchange.

Here is a piece of code that embeds all of these assumptions. The program is able to connect to a server, revealing the intricacies of opening a socket and sending information through the socket so that the other side can get the data that you're trying to exchange. The code embeds assumptions that will affect the coupling between your client and the server. These assumptions could change or become invalid: being aware of them and being aware of the coupling that you introduce into your architecture when you write this type of code is what we want to discuss today.

The location address encodes the knowledge about where is the server that we want to connect this client socket to. The part of the code that knows about the location uses the symbolic address of the server. This is the address that we are looking up with the DNS: if you want to switch the address you have to change this string. Two lines later there is a port number (8888) as well, which is part of the address. Everything else is just boilerplate code that you need to write if you want to open a TCP socket using the address as an input, but you don't have to change it unless you want to switch to a different protocol.

The content semantics is basically embedded in the code that is responsible for knowing the meaning of the data that you are exchanging. The 'Solution' is a class that encodes a certain 'String' with a certain color following the interface data model defined elsewhere.

When you transform the 'answer' object into a set of bytes, this is where you decided which representation format to use. The byte array is basically the format for the data that you use to send a message with. The data is a solution that contains a string and a color, and this byte array is the type of the message in which the data should be sent with.

We can recognize the need to perform a transformation between the internal representation of the data (stored in memory within the structure given by the 'Solution' class) and the external representation of the data (also stored in memory as a byte array so that it is ready to be copied along the protocol stack). This 'getBytes()' is the adapter between the inside and the outside, this is where all the assumptions on how the data gets serialized are found. These assumptions need to match the ones of the message parser on the server side.

Highlight the code making these assumptions

Representation
Format

Location
Address

Content
Semantics

```
fetch( "https://asq.click/api", {  
  headers: {  
    'Accept': 'application/json' }  
} )  
  .then( response => response.json() )  
  .then( sol=>{  
    highlight( sol.text, sol.color );  
  })
```

It's very rare that you actually would directly program sockets these days, so let's take a look at a higher level example. This is no longer Java, this is JavaScript. You should still be able to pick the same three aspects.

Also, we can see the code of a client that will contact a Web API using HTTP via the 'fetch' asynchronous JavaScript API. The code will process the response by parsing the JSON string which gets transformed into an object. Two specific fields of the object get passed to the 'highlight' function.

The location address is probably more clear to spot, since it is found in only one place, written as a URI string encoding the absolute address of the API of the Web resource. By default fetch uses the GET method, so we do not have to explicitly configure it.

How does the code control which representation format will be used? There are two places where this assumption is encoded. The first configures the 'Accept' header to ask the server to use a particular media type as representation for the resource being fetched. The second is where the response is transformed into an object. The body of the response is actually received as a string. We parse the string and transform it into the object, assuming the string uses the JSON syntax.

Then we work with the object and we start making assumptions about its content semantics, by extracting the text and the color fields. This code is actually independent of where the content of the object comes from and how it got there.

You can interpret this code also in terms of connectors and adapters. The first fetch call represents the low-level HTTP protocol used to connect this client code with any remote Web resource (here you need to make sure you select the right address). Its result needs to be fed into the specific, local interface 'highlight'. There is a mismatch between the extremely general result (a string) downloaded as the payload of an HTTP response and what the local function requires as input (two specific parameters with a domain-specific

semantics: text and color).

How to solve this mismatch? Use an adapter which can translate the original string into an object, from which the specific fields can be extracted. Does the local call to highlight the solution depend on where its input data parameters came from? No. Does the actual content of the object depend on the server sending the expected content with the correct semantics in the response? Yes.

What if you change the structure of the object to represent a true/false answer? Which part of the code will need to be changed? Can you reuse the fetch and the JSON parse steps without changing them? They are completely orthogonal from the content semantics. Likewise, you could reuse the same highlight function and pass it an object hydrated from a SQL query sent by a object-relational mapper, as long as the schema would match the content that should be in the object.

Addresses, Protocols, Representations and Semantics are always present when we connect together two different pieces of software. They can be controlled, configured and designed independently: we can take the same content with the same semantics and represent it in completely different ways. And if for some reason we need to move the server to a different location, most of this code will not be affected. The question is whether it's a good idea to actually hard code these addresses into your code, or should they be actually read from some separate configuration settings, to keep the code location agnostic.

Understanding Coupling

- Different connectors introduce different degrees of coupling between components.
- Only disconnected components are fully de-coupled.
- What type of assumptions are implied by a connector?
 - component exists
 - component matches interface
 - component deployed at a given location
 - component running and available to be invoked
 - component can freely evolve

Depending on the connector that you use, you will have a different degree of coupling. Different types of assumptions you make will affect the coupling between the components.

As long as components are connected, they are coupled. Sometimes you read about different degrees of coupling: loose coupling, tight coupling. Not all the coupling is the same, but only components that are completely disconnected are fully decoupled.

Coupling is about the impact of what happens on one side to the other side. This impact directly depends on the assumptions that you make when you select and configure a certain type of connector.

The most fundamental assumption that you already do at design time, is the fact that there is a component on the other side that exists and it matches the interface that you have on your side. This is implied just by the fact that you are drawing the line between the boxes.

The edges connecting the nodes of the structural dependency graph of the logical view of your architecture represent an important assumption: each connected node assumes the other side exists and it matches the interface it requires or provides.

When you switch between the design of your system and its deployment, then you also need to know where you will find the component that you depend on. So we assume the component exists and that the component has been deployed in some container and we know where to find it.

When do we discover the existence of a component? When do we know the location? And do we hard code these answers like in the previous examples? This is a string with the IP address that we write into the code and if we want to change the location we have to change the code and recompile it. That's sounds like a strong coupling: a change of location requires a rebuild. Instead, we should be able to separate the information and the knowledge about the location of our dependencies from the code that actually uses them.

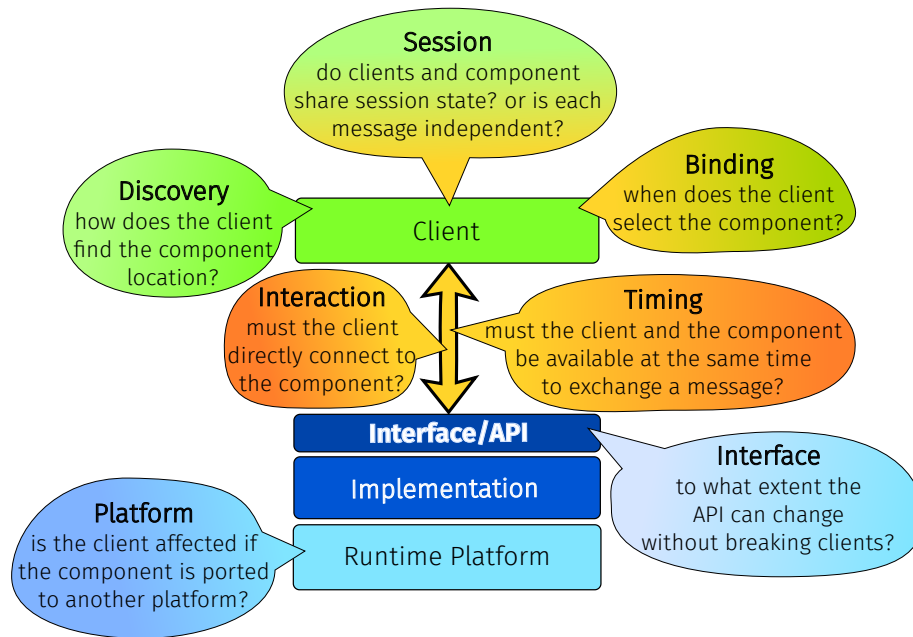
After the deployment we start the system; its components start running. That's when components start to interact with each other. Components that need to interact with each other, depending on the connector that you choose, may assume that the other side will be available to answer their invocations. Or maybe with different connectors, a component will take advantage of the opportunity to actually interact with another component that is not available at the same time. Depending on the connector, the state of the availability of one component may affect the success of the interaction with the other component.

While you operate your system something will fail. How does a failure of one component affect the other side? Will the failure of a random component which you didn't know existed bring down your component? Again, this depends on the type of connector, which can either propagate the failure or isolate it.

As you evolve the system, you make a new version, you ship a new release and deploy it. Components you depend on have a life of their own, they change. Components that used to work, stop working after upgrading them. Interfaces which used to match, are no longer perfectly matching. Which changes on one side are going to be visible and affect the other side?

When you connect together two components, you establish a dependency. You couple them together. And that means they are no longer independent from each other, they cannot change independently. Changes of location, state, availability state, implementation as well as interface will propagate in different ways along coupling relationship introduced by different connectors.

Coupling Facets



Let's see more precisely these different coupling facets. Let's decompose the definition of what is the coupling between two different components into multiple facets.

Here we can see that there is a client that is connected to a certain interface. The interface has an implementation, which runs in a certain container on a certain runtime environment.

The first question that you should ask yourself when you're about to choose the type of connector to introduce is: should both the client, and the component it depends on, be available at the same time in order to successfully communicate?

The answer can help you to constrain your choice of connector. The timing facet of coupling depends on whether the connector is a synchronous or asynchronous one. If the connector is synchronous, for example: remote procedure call, we assume that when we make the call, the other side is available to answer it within a certain time window. If it takes too long, the client will not receive an answer and typically the call will fail due to a timeout.

The other aspect of coupling is related to the location: How does the client discover the address of the interface? How does the client learn about the location of the other side? The client needs to know where to find the component that it depends on. This can be resolved when you write the code of the client, you hard code the IP address of the server into the client. You write it in a configuration file. For example, when you start the client, the client reads the configuration file and it discovers the address of the server. This could also be something that the user can give to the client. If you open a web browser, the first thing you have to write is the address of the Web server. The browser doesn't know it; it's the user who knows. Here are different approaches to keep the client as independent as possible from the knowledge about the location of the server. The more the client knows, the larger impact a server migration will have.

Another difference separates direct or indirect connections between the two components. For example, the Message bus is indirect because the client is going to send a message into the queue. There is a direct connection between the client and the queue, but the other side does not talk to the client, it just talks to the queue to receive the message once it's ready to be delivered and the server is available to process it. Also in this case your choice of connector will change the interaction facet of coupling.

The binding facet is related to an earlier decision that you make related to the selection of which component you are going to depend on. Once you picked the component you can discover its location. But first you need to know which component you are going to use in the first place. Given a certain interface, we can have multiple implementations. So when do we select the implementation? A binding decision is a decision that you make when you choose the component that you want to connect with. Discovery is different because once you have chosen the component once, you know who you want to talk to, then you have to know where to find it. The same component can be located in different places. You may want to search for the nearest location to minimize latency among a globally replicated deployment. The binding is done based on the functional requirements, the discovery helps to optimize the extra-functional requirement of performance and availability.

What if you work with this component and all of a sudden they tell you that from now on you have to start paying 10 times as much if you want keep calling the component. Here it doesn't matter where the component is located. What you want is to switch to a cheaper provider and change your binding (assuming you can find a replacement). As a consequence you will also have to change the location. We can see the location as a more technical facet related to having a distributed runtime and components need to communicate over the network. The binding is more of a business decision which establishes a relationship with a given provider.

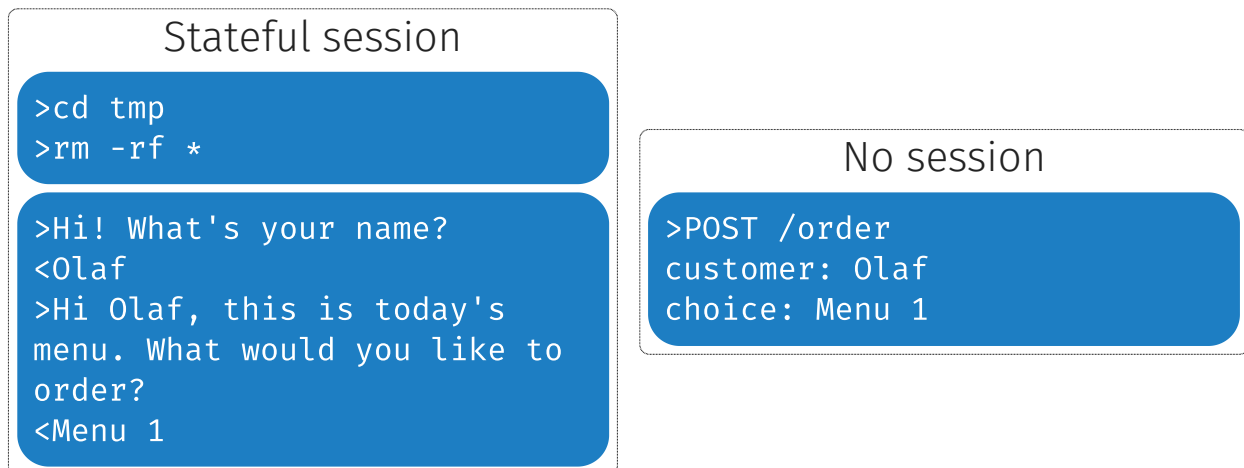
Platform independence is a facet which determines whether changes in the runtime environment of the container in which one component is deployed will affect the other side. For example, consider if you switch the programming language used to implement a component: is the client going to notice? This is usually noticeable concerning the data model of an interface and how data is represented in different languages. If you expose a binary representation which directly maps the memory layout of your objects, you may notice even if you switch to a different compiler version of the same programming language. If you use programming language independent standards, such as JSON, XML, then there are already enough standard adapters between most programming languages and these representations so that your interface becomes platform independent. This way, changes to the dependencies of your dependencies are abstracted away and will not impact whoever depends on you.

Another critical coupling facet concerns the assumptions the client makes about the content of the interface that it depends on. We can pick the right connector to mitigate changes to the location, representation format, timing. We can even rewrite the implementation using a different programming language. And still keep the client compatible, thanks to information hiding. But what if you deprecate or eventually remove some interface feature? The client would be totally destroyed, as it's trying to use something that is not there anymore. We will study in the lecture about flexibility what kind of changes we can do to an API that do not affect clients and what kind of changes you should never do to an API unless you want to break all of your clients.

The session facet depends on type of interaction between the two sides. If the client performs an operation on the interface, will the result of the operation depend on the previous interactions of this client? This is similar to stateful interfaces, for which the results depend on the history of the previous interactions of all clients. Establishing a

session between the client and the interface means that interactions are no longer independent from previous ones. The interface will remember and the client will assume the interface remembers. What could possibly go wrong? Let's see an example.

Session Coupling Examples



Let's start from the shell command example: How many of you would want to run the second command independently of the result of the previous command? The first shell command sets the current directory, which will definitely determine the outcome of the second command. That's why sometimes it is called a shell session: every command will change the state of the shell and you enter the next command assuming the shell is in a certain state. When your assumption fails to match the state of the shell, that's when you could end up deleting your entire file system. Don't try this at home. Always `'pwd'` and check the result before doing a `'rm -rf *'`.

The chatbot example shows a conversational session. Each party will exchange multiple messages in a certain order. And the big question is whether they expect to remember previous messages or whether each message is treated independently of the previous ones. Will a previous message affect the behavior of how further messages are processed? Is it even possible to send a message without having sent another message beforehand?

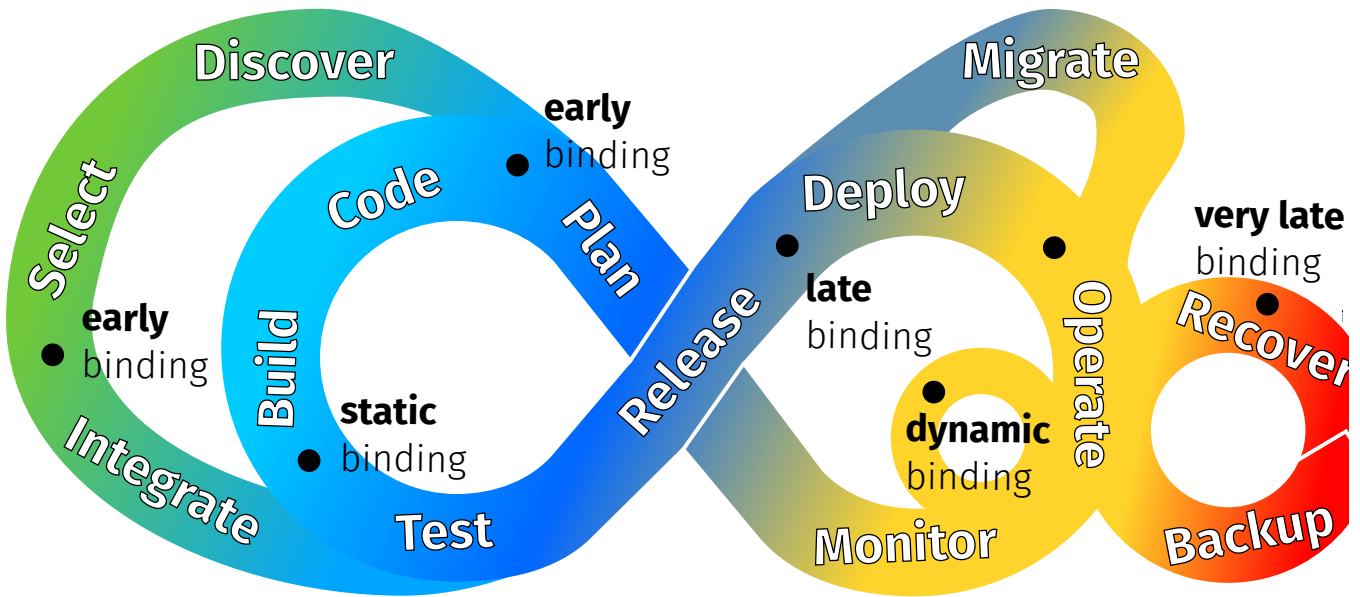
The advantage of chatbots is that they are supposed to be more human friendly, more natural. You feed them information step by step. On the other side, the information gets accumulated until you get to the point in which you want to place the order and commit the transaction. While it makes perfect sense to have this conversation within a few seconds, but how long is the waiterbot willing to wait to have the second part of the conversation. Human short term memory tends to expire after a brief time, this is not the case of bots, which would gladly keep waiting for you to complete the order forever. But if you resume the conversation months later: Do you assume that the state on the other side of the session is still there? How likely is that? Have you ever experienced a long hiatus in a personal relationship but then resumed the conversation decades later as if no time had passed? In this case, the protocol should include mechanisms for checking whether the state of the session is still in synch between the two sides. There is nothing worst than entering data into a complex Web form and getting a session timeout error.

Speaking of Web forms, the third example is about making an HTTP request to place an order. The request here contains both the selection for the menu but also the name of the customer. With one command we get the job done. This is an example of an interface designed to work without establishing a session.

If you have interfaces which require multiple rounds of interactions and every interaction builds on the previous ones, the longer the session takes, the higher the chances that something may go wrong and there may be a partial failure: a failure in which one of the two sides of the interaction loses the state of the interaction, while the other still remembers it. This makes the session state that you have established across the connection inconsistent.

The risk of partial failure and the complexity of recovering from it makes session-ful connectors more coupled. Session-less connectors can simply retry failed interactions since both sides are always in sync as their session state by definition gets reset after every interaction.

Binding Times



Binding Times

- Depending on the connector, the topology of the graph of connected components may be defined, constrained, refined or modified at different points during the system lifecycle
 - design-time (as components are selected)
 - build-time (as components are packaged together into deployable units)
 - deployment-time (as components are unpacked and installed)
 - or even modified at run-time (to optimize the behavior of the system, e.g., switching between service providers).
 - during recovery, when something fails and needs to be replaced

When do you make the selection of the components that you want to reuse and will need to depend on? This can happen throughout the whole lifecycle of your architecture.

Early binding indicates a choice made before running the system. A choice made based on the documentation of the interface we have discovered. We select it, but we still haven't been putting it in action. An early decision is made before you even compile the code.

Static binding also happens before starting up the system. This could be done both for components that you built yourself, or components that you have selected to be integrated from somewhere else.

After you built your system, then you can run tests with it, this is where binding decisions become more dynamic.

Binding after you start to run the system is also known as late binding. For example, after you ship a release, during its installation one may choose which component gets deployed. Sometimes deployment is atomic, all the binding decisions have been made statically and you can only drop the entire package or none. Sometimes you can customize which components get plugged in or out of the system at deployment time. You have already built the system, it's already tested and released, and now during deployment is the time to do the customization.

Dynamic binding happens while the system is running. It requires support for dynamic loading (and unloading) of components without the need for restarting the system. This can be useful to procrastinate the decision about which component to load until the very last moment right before you actually need to call it. Until you're about to make the call, you can still switch the implementation targeted by the call.

If you call something and it fails, it means that your binding decision on what to call was misguided. Maybe it used to work, but for this particular call it didn't because your dependency crashed. If, as part of your recovery, you decide to change your selection of the component that you're going to use, you will re-evaluate the binding doing what we call very late binding.

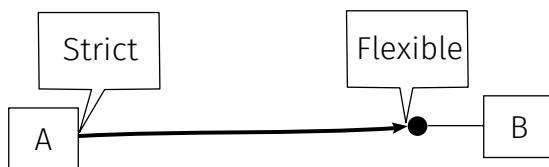
The difference between late and very late binding is that late happens during deployment or startup; very late happens after a failure during recovery, which may involve another startup cycle. As part of recovery, to stabilize your system, you decide not to depend anymore on a flaky provider that is unreliable. You found a better provider or you had a back up provider and you can change the binding.

The more dynamic your binding, the later you wait, the more flexible your system is going to be.

You make early binding decision because you prescribe a connection between the two components based on their documentation. As you build your system, you can statically bind components while packaging a release artifact. Static linking results in a single artifact, which can be atomically deployed. Dynamic linking supports swapping or adding more components at runtime, while the system is operating, making decisions based on information only available at runtime (e.g., performance/workload/resource utilization/-cost).

The earlier they are bound together, the stronger the coupling between components and the more difficult it will be to replace some of them without having to go again through the entire lifecycle (e.g., rebuild or reboot) for your system.

Be liberal in what you accept, and conservative in what you send.



We can conclude this lecture on compatibility and coupling with two messages: some food for thought.

The first concerns the compatibility of two connected components. For example, they're supposed to comply with a standard interface. How do you check or enforce this decision? You have two options: to be as strict as possible in how each side follows the standard, e.g., how it represents the data being exchanged. The other option is to be as flexible and forgiving as possible while exchanging information, which may deviate from the standard, but still work.

One side sends a message to the other. Which side should be strict and which side should be lenient? Having both strict sides is not practical. It would imply having only a single interpretation (or implementation) of the standard. Flexibility on both sides is a recipe for chaos, since there would be too many possible variations from the agreed upon interface.

You can have components producing noisy colorful messages, and strict components rejecting most of them. This also does not make it easy to increase the size of the system while keeping most components compatible.

The only viable alternative for large-scale interoperability is to be forgiving as you accept messages from unknown senders, but being strict while sending messages to the rest of the world. This principle helped to support the growth of the Internet protocol, as it recognizes that while there can be many interpretations of a standard, it is important to provide incentives for compliance.

Water or Gas Pipe?



Sometimes incompatibility is the goal

The second message is that in some cases compatibility can be a problem.

Like in this example, where we have two identical pipes but we want to reuse them to carry different content: water and gas. This makes it more efficient to source the pipes while constructing the building. However, how can you avoid connecting them together? After all, you want to prevent starting a fire when opening your bathroom faucet.

The solution is simple: make the interfaces incompatible so that they cannot be connected together by mistake. You just need a standard or to pick a convention where gas and water pipes are threaded in the opposite direction to avoid they fit together for safety reasons. Safety can justify incompatibility, and you should not try to work around it with some adapter.

Incompatibility can also be a business strategy to create vendor lock in, keep customers captive within your walled garden as it becomes too expensive to switch to alternative platforms. Incompatibility can fuel the growth of an industry dedicated to producing adapters.

References

- Richard N. Taylor, Nenad Medvidovic, Eric M. Dashofy, *Software Architecture: Foundations, Theory and Practice*, John-Wiley, January 2009, ISBN 978047016774
- Gregor Hohpe and Bobby Woolf, **Enterprise Integration Patterns**, Addison-Wesley, October 2003, ISBN 0321200683
- Douglas C. Schmidt, **Half Sync/Half Async**, 1999
- Cesare Pautasso, Erik Wilde, **Why is the Web Loosely Coupled? A Multi-Faceted Metric for Service Design**, pp. 911-920, Proc. of the 18th International World Wide Web Conference (WWW 2009), ACM, Madrid, Spain, April 2009.
- Jon Postel (ed.), **TRANSMISSION CONTROL PROTOCOL**, RFC 761
- Cesare Pautasso, Gustavo Alonso, **Flexible Binding for Reusable Composition of Web Services**, Proc. of the 4th Workshop on Software Composition (SC 2005), Edinburg, Scotland, April 2005
- XKCD 927: **Standards**
- Klint Finley, **The Oracle-Google Case Will Decide the Future of Software**, Wired, 23 May 2016

Software Architecture

Deployability, Portability and Containers

9

Contents

- Deployability Metrics
- To Change or not to Change
- Continuous Integration, Delivery and Deployment
- Platform Independence: Virtualization and Containers